1. **Project Information**:

Title: Real Time Ray Tracing with WebGPU
Team members: Youssef Gaballa (Solo)
Selected Project: Propose your own (similar to Ray Tracing in one weekend):
This project aims to create a realtime ray tracer using WebGPU.

2. **Project Description**:

I built a ray tracer that runs on the GPU with options to customize the scene, camera, lighting, and the ray tracing Bounding Volume Hierarchy (BVH). It allows controlling camera position and rotation when the canvas is clicked. W,A,S,D keys move the camera along the x-z axis, space move it up, C or ctrl moves it down.
It has two scenes: each scene has a square floor (represented as two triangles) and a number of spheres. Scene 1 has six spheres with different properties and positions. Scene 2 can allow a variable number of spheres- it has a sphere count slider to control the number of spheres, each with random properties. There is also a controls panel on the left that allows controlling the position, radius, color, and material properties of each sphere, as well as the width, color, and checkerboard pattern for the floor (quad 0). The floor is also customizable: can switch between checkerboard and flat color. The UI can also show the FPS (frames per second) and ms per frame (SPF = 1 / FPS). One can also toggle the bounding hierarchy visual, which shows the actual hierarchy of bounding boxes color coded based on depth, with boxes at the root being red, and leaf nodes being green, and nodes in between root and leaves interpolated between (red to orange). Also, one can toggle on temporal accumulation which stores cumulative frame buffers so that when averaged out in the compute shader, it removes the noise.

3. **Steps Taken**:

I first wrote the code to initialize WebGPU by finding a compatible device and driver that supported WebGPU. After there were no errors, I then created a triangle to test the functionality of WebGPU in different browsers.
Then I wrote the compute shader that can run the ray tracing algorithm. In this ray tracing algorithm, it sends many rays out from the camera. When rays intersect a sphere, the material properties of the sphere that the ray hits is recorded into a hit record that the function returns. As these rays intersected objects, the ray either spawned a transmission ray / reflected ray (for transparent materials), or a reflected ray (for metallic materials), or a diffuse ray (for matte materials). This continues up until the max number of bounces has been reached (which can be set in the UI). One should note that for metallic materials, the scattered direction is interpolated between the reflect direction and diffuse direction based on the reflectivity- if reflectivity is 1, it scatters in the reflective direction, if it is 0, it scatters in the diffuse direction. For refractive materials, I also had to consider the case when a ray enters a medium with a lower index of refraction- if it enters at an angle less than 90 degrees, then there is no solution for Snell's law. In this case, the ray is interpolated between the reflected direction and diffuse direction. This case does not show up since the index of refraction of air is 1.0, and the objects are restricted to an index of refraction greater than 1.0. Rays that did not hit a light source but went off to infinity had the pixel rendered using the background color / texture (skybox).
Bounding Volume Hierarchies (BVH) are an acceleration structure that encompasses several objects and speed up calculations by skipping hierarchies that rays did not intersect. In WebGPU, compute shaders did not support function recursion since function calls needed to be

resolved to a fixed size call stack at compile time, so I needed to find a workaround. This workaround used a stack to replicate recursion. Basically,

4. **Challenges**:

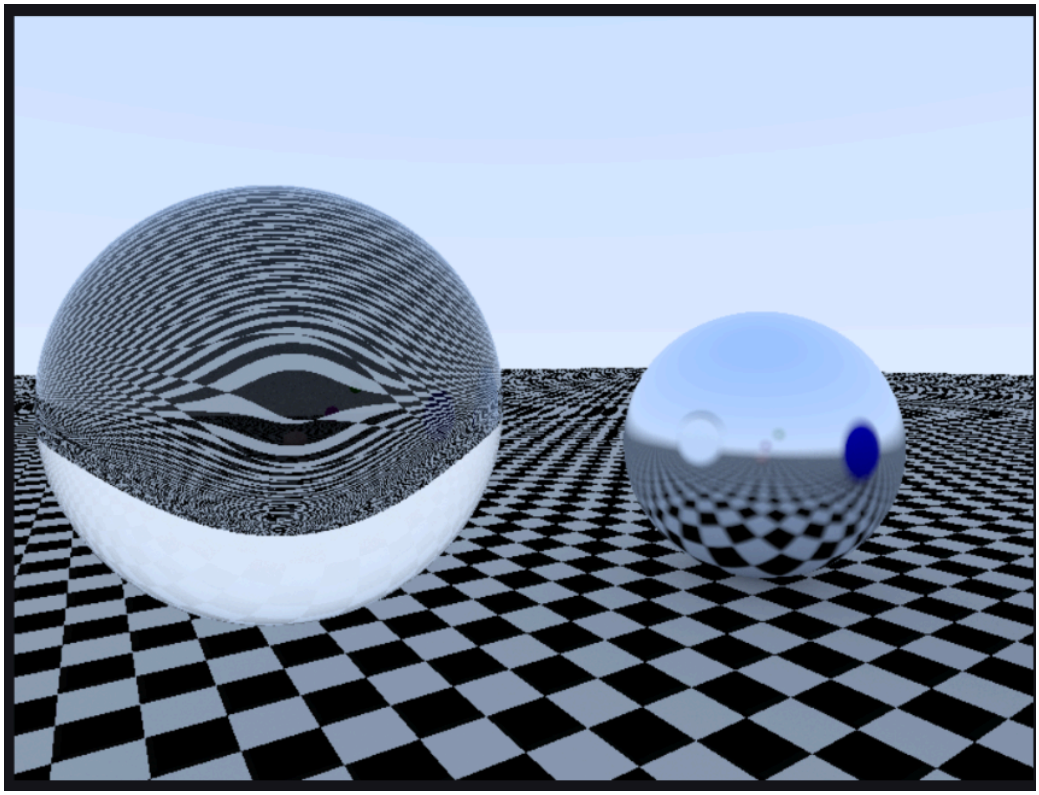
One challenge was creating the BVH according to the Surface Area Heuristic (SAH). There are other ways of constructing the BVH that are easier, but inefficient for the traversal algorithm. I picked the SAH because it is used more often in practice in actual ray tracers, so I thought it would be good to get familiar with that algorithm. I did a lot of research into the algorithm (see resources that I used in the bottom of this paper) and did some experimenting with visualizing the BVH boxes to see if they were splitted according to the SAH correctly. The algorithm is the most efficient since it creates a BVH where each node's AABB is constructed so that it has the smallest possible surface area.. This way, rays that are more likely to miss an object will also miss the AABB. For the algorithm to work, I had to loop over each of the x, y, and z axes and create potential left and right child bounding boxes for each child box that the current node contains (means having an inner loop over the span of objects it contains). Then I keep track of the minimum cost and the axis and object index associated with that cost. At the end of the loop, the sphere indices are sorted so that the objects to the left of that minimum object index along the best axis come before it in the array, and objects to the right come after.Then we do the same thing for the left and right child recursively.

One of the hardest challenges was trying to figure out why the bounding volume hierarchy traversal is slower than looping over each sphere and testing if the ray hits the sphere, if I could make it faster. I had to do some research on GPUs and found out that there is something called SIMT (Single Instruction, Multiple Threads) where each thread executes the same instruction The, but with different data.The GPU automatically optimizes the loop over each sphere so that each thread can process the hit function for every sphere in parallel, but it can’t do the same thing for the stack based algorithm for the BVH traversal since the stack pointer is not always incremented by 1 in actually depends on whether or not the bounding box we hit has two more children or is a leaf node. I tried to overcome this by creating an alternative type of BVH called a stackless BVH, where instead of having a left and right child, there is instead only 1 child (left child) and a skip pointer that the traversal algorithm follows if it misses. The skip pointer points to the parent node’s right child so that if it misses that left child, it goes to the parent’s right child and tries again. If it misses every node, it hits the sentinel node (which is not an actual node) and ends the loop (resulting in a miss). This algorithm was slightly faster, but still not fast enough so it was dropped from the implementation.

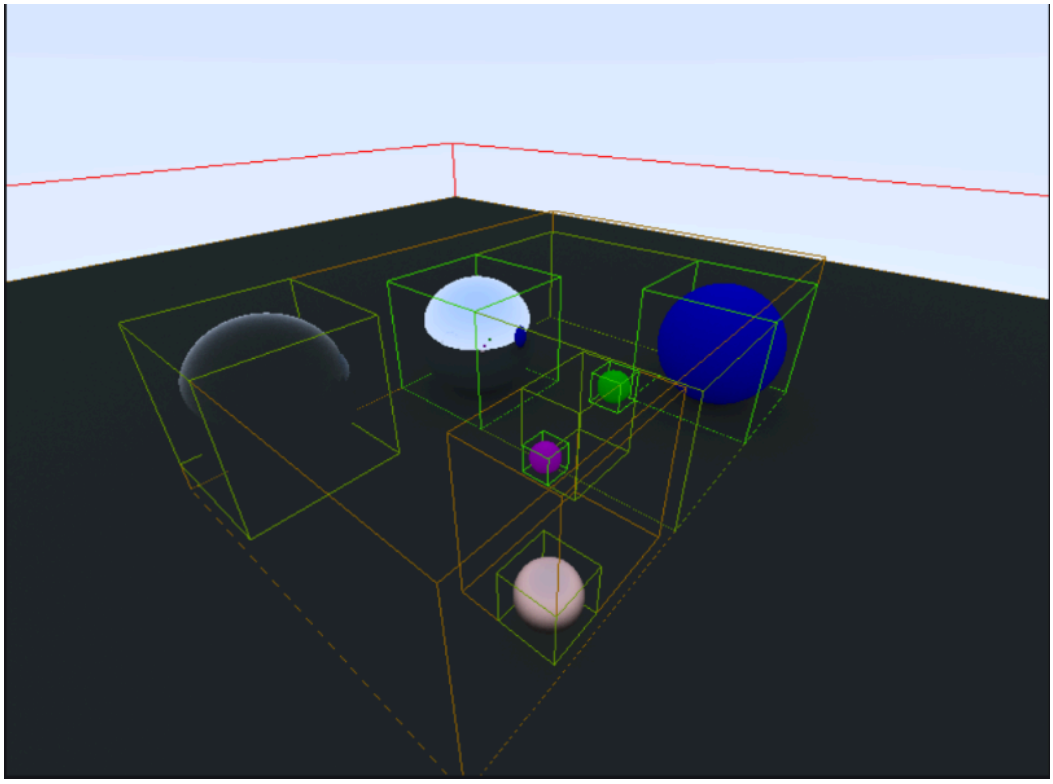
5. **Results**:

My implementation works very well with a manageable number of spheres. At 5 spheres the fps is over 2000 (on my machine), and only drops below 100 fps at around 360 spheres. I found that the number of ray bounces doesn't make much of a visual difference as long as it is above 3. Toggling between the simple diffuse lighting calculation (which is less realistic) and the Lambertian calculation makes the small shadow at the base of the sphere larger. Moving each of the spheres recalculates the BVH so that the visual is updated. The accumulate frames button also shows what it looks like if the accumulation buffer interpolation isn't reset if the

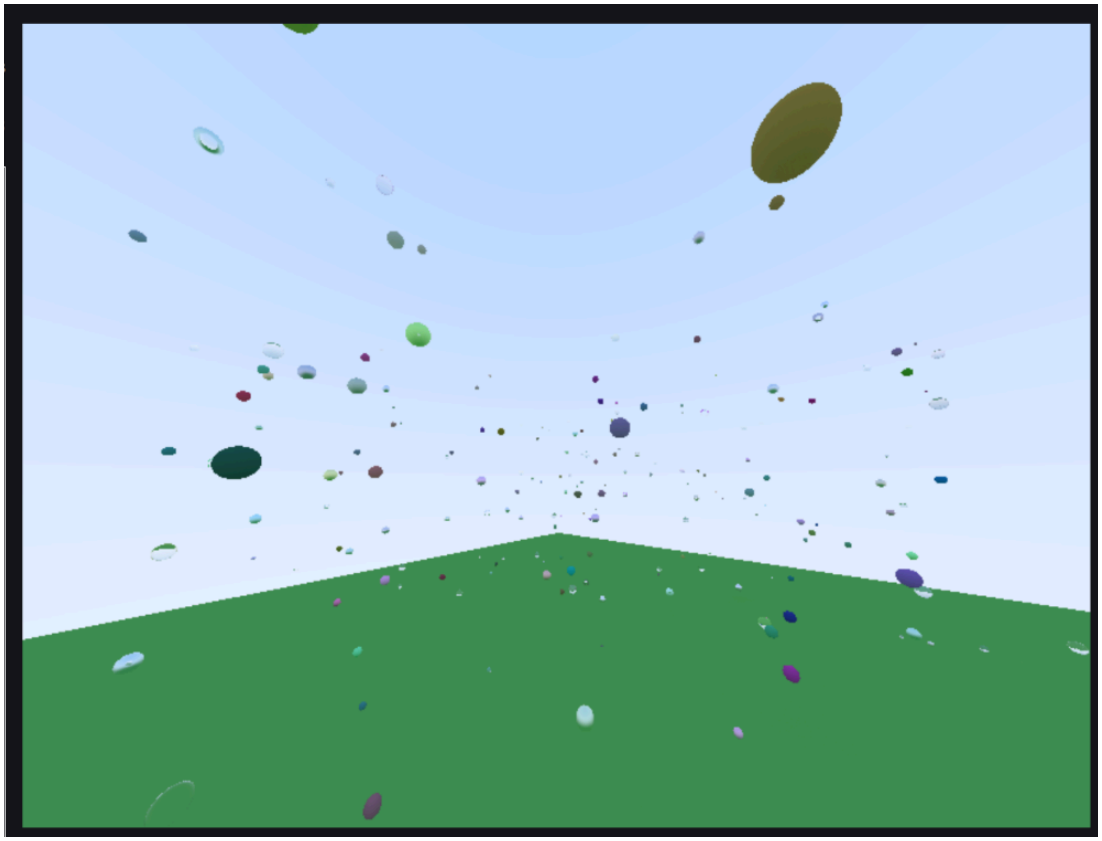
camera moves or if a visible object moves (it still keeps the old buffer without reinterpolating).
There is still plenty that could be improved in my opinion:
I could add textures, specular highlights, better denoising, movable objects, and distributed ray tracing effects like motion blur, depth of field.
Here are some screenshots:
Refracted and metallic sphere:



BVH visual:



Many spheres:



6. **Effort Justification**:

This project represents the 1.5x (and over) effort of a regular homework assignment because I am using a new API (WebGPU) and new shading language (WGSL). I have never used either before, so a lot of time was spent learning how WebGPU and WGSL work.
I also implemented real time ray tracing, which is typically harder than offline ray tracing (used in the Ray Tracing in one weekend book) since it not only has to implement ray tracing correctly but also be performant. I dealt with complex algorithms like BVH construction and BCH traversal. I also used a lot of my knowledge on matrices in using the projection matrix and inverse projection matrix so that the scene is synchronized with the BVH visual boxes (which need a separate pass in a vertex and fragment shader to be rendered. I also had to do a lot of research into ray tracing algorithms and how light works (I learned about Snell's law). I also learned a lot about how GPUs work in compute shaders in that some code may not run as fast as you think. Also, the most common issue I faced whenever I added a new feature was that when I updated the struct in the shaders, I also had to make sure the byte alignment was correct, since WebGPU has strict rules for alignment. For example, while a vec2f is aligned to 8 bytes (2 floats is 4 * 2 = 8 bytes), and a vec4f is aligned to 16 bytes, a vec3f is aligned to 16 bytes and not 12 bytes. I also had to keep in mind that implicit padding is added if my alignment is off. Also, I had to update the buffer on the CPU side correctly and made sure that if it needs to be updated dynamically, the update buffer function (and configure bind groups) is called in the update function in the Renderer class.
I also I did a lot of work on UI, so while that part may not have the hardest problems to solve, it did take a while since it is a lot of boilerplate.

7. **Tools and Resources:**

WebGPU: webgpufundamentals.org , w3.org/TR/webgpu
WGSL: google.github.io/tour-of-wgsl
Research Papers: Ray Tracing Complex Scenes (Kay & Kajiya, 1986), Distributed Ray Tracing (Cook et al., 1984).
Paper on SAH: MacDonald, J.D., Booth, K.S. Heuristics for ray tracing using space subdivision. The Visual Computer 6, 153–166 (1990). https://doi.org/10.1007/BF01911006
SAH Articles:
medium.com/@bromanz/how-to-create-awesome-accelerators-the-surface-area-heuristic-e14b5dec6160
jacco.ompf2.com/2022/04/18/how-to-build-a-bvh-part-2-faster-rays/
Helpful WebGPU playlist:
www.youtube.com/playlist?list=PLn3eTxaOtL2Ns3wkxdyS3CiqkJuwQdZzn

AI - Google Gemini:
My AI use was mainly educational/for research. I use gemini since it is the only AI model that I know of that actually cites resources which makes it helpful for researching. The instructions don't ban AI use and the course page says it is permitted as long as I declare its use. I used gemini to help me understand and debug the BVH node process, as that process was taking way too long and was taking time away from actually coding some other ray tracing features.

For some reason the BVH was slower than simply looping through the array of spheres, and I knew it had something to do with SIMD on the gpu, but I wasn't sure what. Since I didn't know very much about GPU internals, and that researching that topic would take too long and wouldn't provide much benefit, I asked Gemini if it knew why it was slower and gave an answer that didn't really help much.